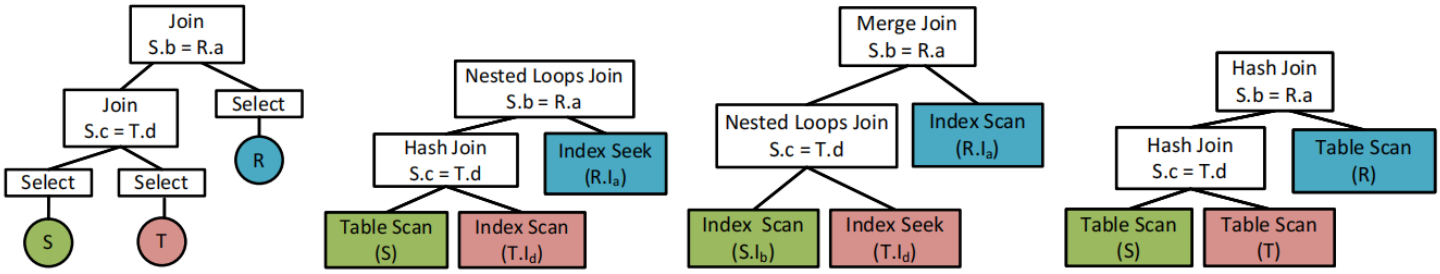
JOIN 算法分析

主要包括三种： Hash Join 、 Nested Loop Join 和 Merge Join 算法的原理。

1. SQL 语句

```
SELECT *
FROM R, S, T
WHERE R.a = S.b AND S.c = T.d AND T.e = 10
```

生成的逻辑计划树和可能的物理执行计划：



(a) Logical plan L **(b)** Execution plan P_1 **(c)** Execution plan P_2 **(d)** Execution plan P_3

Figure 1.3: Different execution plans for a given logical query tree

1.1. 查询分析

这是一个三表连接查询，包含以下关键组件：

1. 投影操作（ Projection ）
   - 具体含义：返回表 R ， S ， T 的所有列（ R.* ， S.* ， T.* ）。
2. 连接操作（ Join ）
   - 连接条件：
     - R.a = S.b （表 R 和 表 S 的连接）；
     - S.c = T.d （表 S 和 表 T 的连接）；
   - 连接类型： INNER JOIN ；
   - 连接选择性：连接结果的大小取决于连接列的选择性；
   - 连接模式：星型连接模式，表 S 作为中心表连接 R 和 T 。
3. 选择操作（ Selection ）
   - 选择条件： T.e = 10 （对表 T 的过滤）；

- 谓词类型：等值选择谓词；
- 应用时机：应尽早应用（谓词下推）以减少中间结果大小。

该查询推荐的等价显式 `JOIN` 语法：

```sql
SELECT R.*, S.*, T.*
FROM R
    INNER JOIN S ON R.a = S.b
    INNER JOIN T ON S.c = T.d
WHERE T.e = 10
```

1.2. 逻辑计划树

根据图 1.3(a) 表示的逻辑计划树结构：

```
Join (S.b = R.a)
├── Join (S.c = T.d)
│   ├── Select (S)
│   └── Select (T)
└── Select (R)
```

执行逻辑：

- 根节点是 `Join S.b = R.a` （`S` 和 `R` 的连接）；
- 左子树是 `Join S.c = T.d` （`S` 和 `T` 的连接）；
- 叶子节点是三个表的选择操作： `Select(S)`、`Select(T)`、`Select(R)` ；
- 逻辑上，先执行 `S` 和 `T` 的连接，然后将结果与 `R` 连接；
- 所有三个表都有 `Select` 操作，表明可能都需要应用选择谓词或投影操作。

2. 连接算法详解

2.1. `Hash Join` （哈希连接）

1. 算法原理
   - 基本思想：使用哈希表来加速连接操作。
   - 执行过程：
     a. 选择较小的表作为**构建表**（build table）；
     b. 为构建表创建哈希表；
     c. 扫描**探测表**（probe table），在哈希表中查找匹配记录。

2. 执行计划分析（图 1.3d）
   根据图示的实际执行计划：

```
Hash Join (S.b = R.a)
├── Hash Join (S.c = T.d)
│   ├── Table Scan (S)
│   └── Table Scan (T)
└── Table Scan (R)
```

   执行流程：
   i. 扫描表 `S`，构建哈希表（基于 `S.c` 作为哈希键）；
   ii. 扫描表 `T`，在哈希表中查找 `T.d` 的匹配项，完成 `S` 和 `T` 的连接；
   iii. 根据 `S` 和 `T` 连接的结果，构建哈希表（基于结果中的 `S.b` 作为哈希键）；
   iv. 扫描表 `R`，在哈希表中查找 `R.a` 的匹配项，完成最终连接。

   关键特点：
   - 两个 `Hash Join` 都使用全表扫描，不依赖任何索引；
   - 需要足够内存来构建两个哈希表；
   - 适合处理大数据集，特别是当没有合适索引时；
   - 构建表的选择基于表的大小，通常选择较小的表作为构建表。
3. 应用场景
   - **最佳场景**：适合 `S` 和 `T` 是较大的数据表且在连接键上无索引支持时。
   - 关键条件：
     - 内存充足：需要足够的内存来构建哈希表；
   - 性能优势：
     - 不依赖任何索引，完全基于全表扫描；
     - 对于大型数据集连接非常高效。
4. 性能
   - **时间复杂度**：`O(M + N)`（平均情况）
   - **空间复杂度**：`O(min(M, N))`（需要为较小表构建哈希表）

2.2. Nested Loop Join （嵌套循环连接）

1. 工作原理
   - 基本思想：对于外层表的每一行，扫描内层表的所有行来寻找匹配的记录。
   - 执行过程：
     a. 选择一个表作为外层表（驱动表）；
     b. 对外层表的每一行，扫描内层表；
     c. 如果有索引，可以使用索引查找而不是全表扫描。
2. 执行计划分析（图 1.3b）

根据图示的实际执行计划：

```
Nested Loops Join (S.b = R.a)
├── Hash Join (S.c = T.d)
│   ├── Table Scan (S)
│   └── Index Scan (T.Id)
└── Index Seek (R.Ia)
```

执行流程：

i. 对表 `S` 进行全表扫描（`Table Scan`）；
ii. 对表 `T` 使用索引 `Id` 进行索引扫描（`Index Scan`）；
iii. 使用 `Hash Join` 算法连接 `S` 和 `T`（基于 `S.c = T.d`）。具体地，构建表 `T` 的哈希表，然后在表 `S` 中查找匹配项；
iv. 对 `Hash Join` 的结果集，使用 `Nested Loop Join` 与表 `R` 连接；
v. 对表 `R` 使用索引 `Ia` 进行索引查找（`Index Seek`），寻找 `S.b = R.a` 的匹配。

关键特点：

- 这是一个混合策略的执行计划；
- 内层连接（`S` 和 `T`）使用 `Hash Join` 算法；
- 外层连接（`Hash Join` 的中间结果和 `R`）使用 `Nested Loop Join` 算法；
- 充分利用了 `T.Id` 和 `R.Ia` 索引来提高查找效率。

3. 适用场景
   - **最佳场景**：当 `S` 和 `T` 连接后的结果集较小，且 `R.a` 上有可用索引时。
   - 关键条件：
     - 连接结果大小：`S` 和 `T` 连接的结果集较小；
     - 索引可用性：在连接列 `R.a` 上有可用的索引 `Ia`，支持高效的索引查找；
   - 性能优势：
     - 内层 `Hash Join` 高效处理 `S` 和 `T` 的连接（适合中等规模数据）；
     - 外层 `Nested Loop Join` 充分利用 `R` 表的索引 `Ia` 进行快速扫描；
     - 当 `S-T` 连接结果较小时，外层循环次数少，整体性能优异；
     - 组合策略在特定条件下可能是三种方案中最高效的。
4. 性能
   - **时间复杂度**：`O(M × N)`，其中 M 和 N 分别是两个表的行数
   - **空间复杂度**：`O(1)`（不需要额外的大量内存）

2.3. Merge Join（归并连接）

1. 算法原理
   - 基本思想：将两个已排序的表进行归并操作。
   - 执行过程：

a. 确保两个表都按连接列排序；
b. 同时扫描两个表，比较连接列的值；
c. 当找到匹配时，输出结果。

2. 执行计划分析（图 1.3c）
根据图示的实际执行计划：

```
Merge Join (S.b = R.a)
├── Nested Loops Join (S.c = T.d)
│   ├── Index Scan (S.Ib)
│   └── Index Seek (T.Id)
└── Index Scan (R.Ia)
```

执行流程：

i. 使用索引 `S.Ib` 对表 `S` 进行索引扫描（`Index Scan`）；
   - 关键分析：虽然连接条件是 `S.c = T.d`，但优化器选择了 `S.Ib` 索引（在列 `b` 上）。
   - 可能原因：`S.Ib` 是**覆盖索引**，包含了列 `c` 的信息（Included Columns）。

```sql
-- S.Ib可能是一个覆盖索引
CREATE INDEX S_Ib ON S(b) INCLUDE (c);
```

   - 排序考虑：外层 `Merge Join` 需要 `S.b` 的排序结果，使用 `S.Ib` 索引可以直接提供这种排序。

ii. 使用索引 `T.Id` 对表 `T` 进行索引查找（`Index Seek`），加速表 `T` 的扫描；
iii. 使用 `Nested Loops Join` 算法完成 `S` 和 `T` 的连接 `S.c = T.d`；
iv. 使用索引 `R.Ia` 对表 `R` 进行索引扫描（`Index Scan`），确保 `R.a` 按排序顺序处理
v. 使用 `Merge Join` 算法将 `S-T` 连接的结果与 `R` 表进行归并连接（`S.b = R.a`）

关键特点：

- 外层连接使用 `Merge Join`，要求 `S.b` 和 `R.a` 的数据按排序顺序处理
- 内层连接使用 `Nested Loops Join`，通过索引高效查找匹配记录
- 索引 `R.Ia` 必须提供 `R.a` 的排序顺序，以支持归并连接的排序要求
- 索引 `S.Ib` 支持高效的索引扫描，索引 `T.Id` 支持高效的索引查找
- 这是一个混合策略的执行计划，结合了 `Nested Loop Join` 和 `Merge Join` 的优势。

3. 适用场景
   - **最佳场景**：当 `S.b` 和 `R.a` 上都有排序索引可用时。
   - 关键条件：
     - 索引可用性：在 `S.b` 上有可用的索引 `Ib`，在 `R.a` 上有可用的索引 `Ia`；
     - 排序要求：这些索引能够提供 `Merge Join` 所需的排序顺序；
     - 数据特征：连接列上的数据分布适合归并操作。
   - 性能优势：

    - 内层 `Nested Loop Join` 充分利用索引 `T.Id` 进行高效查找；
    - 外层 `Merge Join` 利用索引提供的排序顺序，避免额外排序开销；
    - 在有合适索引支持的情况下，可以实现接近线性时间复杂度；
    - 索引的排序特性使得归并连接操作高效且稳定。
4. 性能
    - **时间复杂度**： `O(M + N)` （假设数据已排序）
    - **空间复杂度**： `O(1)` （不需要额外的哈希表）

3. 关键效率因素总结

3.1. 连接大小（Join Size）

核心原理：连接大小是指连接操作产生的行数，是选择最优执行计划的关键决策因素。

1. 小连接结果的优势

- `Nested Loop Join` 效率最高：当 `S` 和 `T` 连接产生的行数较少时；
- 外层循环成本低：循环次数少，总体执行时间短；
- 索引查找效率高： `R.a` 上的索引 `Ia` 能够快速定位匹配记录；
- 内存压力小：不需要构建大型哈希表。

2. 大连接结果的特点

- `Hash Join` 表现最佳：当 `S` 和 `T` 连接产生的行数较多时；
- 线性时间复杂度： `O(M+N)` 的时间复杂度不受连接结果大小影响；
- 并行处理能力：可以充分利用现代处理器的并行特性；
- 内存需求大：需要足够内存构建哈希表。

3.2. 索引的作用

1. 排序索引

- `Merge Join` 的基础： `S.b` 和 `R.a` 上的索引提供排序顺序；
- 避免额外排序：直接利用索引的排序特性；
- 稳定性能：不受数据分布影响的一致性能。

2. 查找索引

- `Nested Loop Join` 的加速器： `R.a` 上的索引 `Ia` 支持快速查找；
- 随机访问优化：将 `O(N)` 的扫描优化为 `O(log N)` 的查找；

- 选择性影响：索引选择性越高，效果越显著。

4. 性能比较总结

算法	时间复杂度	空间复杂度	最佳场景	索引依赖
Nested Loop	O(M×N)	O(1)	小结果集	内层表有索引
Merge Join	O(M+N)	O(1)	有序数据	连接列有索引
Hash Join	O(M+N)	O(min(M,N))	大数据集	不依赖索引